

Course Project

Goal

Design, develop and deploy an innovative AI agent that autonomously solves a well-defined niche problem, leveraging state-of-the-art AI models with strong reasoning capabilities and appropriate AI agent architectures.

*Note, choose and develop **one** of the ideas presented.

Requirements

1) Build the agent in an optimized way

Your implementation should be efficient:

- Avoid unnecessary LLM calls.
- Minimize prompt/context size (only what's needed).
- Stay within the project budget.

2) API Endpoints (Required)

Your system must expose the following HTTP endpoints (names must match exactly):

A) `GET /api/team_info`

Returns student details.

- **Purpose:** retrieve student names and emails.

Response format (JSON):

```
{
  "group_batch_order_number": "{batch#}_{order#}", // from
presentation list
  "team_name": "Your Team Name",
  "students": [
```

```
{ "name": "Student A", "email": "a@..." },  
  { "name": "Student B", "email": "b@..." },  
  { "name": "Student C", "email": "c@..." }  
]  
}
```

B) GET /api/agent_info

Returns agent meta + how to use it.

Must include:

- description
- purpose
- prompt templates (suggested way to work with the agent)
- prompt examples and full responses

Response format (JSON):

```
{  
  "description": "...",  
  "purpose": "...", // what this agent purpose  
  "prompt_template": {  
    "template": "..."  
  },  
  "prompt_examples": [  
    {  
      "prompt": "Example prompt 1...",  
      "full_response": "Full response your agent returns..."  
      "steps": [full list of steps, see below]  
    }  
  ]  
}
```

*** Example *:**

```

{
  "description": "An email drafting assistant that turns a short, informal brief into a clear, ready-to-send email draft. The user says who the email is for, what they want to say, and in what tone; the agent returns a structured draft with a subject line and body.\n\nWhat it CAN do: draft new emails from a one-line brief, set or change the tone (formal / friendly / apologetic / neutral), generate a subject line, and write in English or Hebrew.\n\nWhat it CANNOT do (constraints): it never sends email (draft only); it has no access to the user's inbox, contacts, calendar, or any real account data; and it refuses to write deceptive, spam, phishing, or harassing messages. When a request falls outside this scope, the agent returns a short refusal and an alternative suggestion instead of a draft.",
  "purpose": "Cut the time it takes to write routine emails by turning a short brief into a polished, tone-appropriate draft.",
  "prompt_template": {
    "template": "Recipient: <who the email is for, e.g. 'my lecturer', 'a client'>\nGoal: <what you want to achieve, e.g. 'ask for a deadline extension'>\nKey points: <comma-separated points the email must include>\nTone: <formal | friendly | apologetic | neutral>\nLanguage: <English | Hebrew>",
    "example": "Recipient: a teammate\nGoal: ask them to review my pull request\nKey points: PR link is in Slack, I need it before Thursday\nTone: friendly\nLanguage: English"
  },
  "prompt_examples": [
    {
      "prompt": "Recipient: my landlord\nGoal: ask them to fix the broken water heater in my apartment\nKey points: no hot water for 3 days, I already tried resetting the unit, I'm available for a technician on weekday mornings\nTone: neutral\nLanguage: English",
      "full_response": "Subject: Request to Repair Broken Water Heater\n\nHello [Landlord's name],\n\nI'm writing to let you know that the water heater in my apartment has stopped working. There has been no hot water for the past three days. I've already tried resetting the unit, but that hasn't resolved the issue.\n\nCould you please arrange for a technician to take a look? I'm"
    }
  ]
}

```

available on weekday mornings and can be there to provide access.\n\nThank you for your help.\n\nBest regards,\n[Your name]",

```
"steps": [  
  {  
    "module": "IntentAnalyzer",  
    "prompt": {  
      "system_prompt": "You analyze a short email brief and return a plan as a  
JSON object only (no prose). Decide in_scope (false if the request is  
deceptive, spam, phishing, harassing, or not an email-drafting task). If  
in_scope is true, fill: recipient_role, goal, must_include (array of strings),  
tone, language, and a suggested subject. If in_scope is false, set those fields  
to null and fill reason with a short explanation. Do not write the email.",  
      "user_prompt": "Recipient: my landlord\nGoal: ask them to fix the broken  
water heater in my apartment\nKey points: no hot water for 3 days, I already  
tried resetting the unit, I'm available for a technician on weekday  
mornings\nTone: neutral\nLanguage: English"  
    },  
    "response": {  
      "in_scope": true,  
      "recipient_role": "landlord",  
      "goal": "request repair of a broken water heater",  
      "must_include": ["no hot water for 3 days", "already tried resetting the  
unit", "available for a technician on weekday mornings"],  
      "tone": "neutral",  
      "language": "English",  
      "subject": "Request to Repair Broken Water Heater",  
      "reason": null  
    }  
  },  
  {  
    "module": "EmailComposer",  
    "prompt": {  
      "system_prompt": "You write a single email from the given plan. Match  
the tone and language, use the provided subject, cover every item in  
must_include, and keep it concise. Return a JSON object with keys subject  
and body.",
```

```

    "user_prompt": "{ \"recipient_role\": \"landlord\", \"goal\": \"request repair of a broken water heater\", \"must_include\": [ \"no hot water for 3 days\", \"already tried resetting the unit\", \"available for a technician on weekday mornings\" ], \"tone\": \"neutral\", \"language\": \"English\", \"subject\": \"Request to Repair Broken Water Heater\" }",
    "response": {
      "subject": "Request to Repair Broken Water Heater",
      "body": "Hello [Landlord's name],\n\nI'm writing to let you know that the water heater in my apartment has stopped working. There has been no hot water for the past three days. I've already tried resetting the unit, but that hasn't resolved the issue.\n\nCould you please arrange for a technician to take a look? I'm available on weekday mornings and can be there to provide access.\n\nThank you for your help.\n\nBest regards,\n[Your name]"
    }
  ]
}

```

c) GET /api/model_architecture

Returns the architecture diagram as an image (PNG).

- **Purpose:** retrieve a PNG image of the model architecture.
- The architecture must be clear.
- **All sub-modules / sub-agents names must be consistent** across:
 - the architecture diagram
 - your **steps** logging (see </api/execute>)
 - any descriptions you provide

Response:

- Content-Type: image/png
 - Body: the PNG file
-

D) POST /api/execute

This is the main entry point.

- User sends an input prompt.
- Your API returns the agent response + the full traced steps.

Input format (JSON):

```
{  
  "prompt": "User request here"  
}
```

Response format (JSON) – must match exactly these top-level fields:

```
{  
  "status": "ok",  
  "error": null,  
  "response": "...",  
  "steps": []  
}
```

If error:

```
{  
  "status": "error",  
  "error": "Human-readable error description",  
  "response": null,  
  "steps": []  
}
```

Steps:

steps is an array describing **every LLM call** the agent did in order.

You must include:

- module/submodule name (must correlate to architecture diagram)
- prompt
- response

Required step object schema:

```
{  
  "module": "...", // the module name according to your  
architecture  
  "prompt": {  
    "System_prompt": "..."  
    "User_prompt": "..."  
  },  
  "response": {},  
}
```

3) Frontend/GUI (Required)

You must provide a minimal web UI to operate your agent. The gui should be available on your root url: <https://your-url/>

GUI Requirements

- A text input (textarea) for entering a prompt/task.
- A **"Run Agent"** button that calls `POST /api/execute`.
- Display the **final agent response** (`response`).
- Do not have any authentication guards (login, signup, etc.), the gui should be available immediately.
- Display the **full steps trace** (`steps`), including:

- module
- prompt
- Response

Optional (Only if supported by your agent)

- Support back-and-forth interaction (follow-up prompts).
- Display conversation history in the UI.

The UI should be simple and focused on interacting with the agent and inspecting its execution.

Models

- MB5R2CF-azure/gpt-5.4-mini - Text Model
- MB5R2CF-azure/text-embedding-3-small - Embedding Model

Deployment

Deploy your agent on **Vercel** (<https://vercel.com>), Keep your account active until receiving a grade.

****Note (Important), Vercel is a serverless solution, the maximum duration of any api call is 300 seconds (5 minutes), your api calls including `/api/execute` should take less than that.**

Make sure the solution works both on your development environment and production environment.

Databases

- **Supabase:** primary database.
- **Pinecone:** for embedding / vector DB.

LLM Provider (LLMod.ai)

- Each group must create its own LLMMod.ai API key.

- **Note:** In the LLMMod.ai platform, all group members share the same API key, if any member creates or rotates the key, it is updated and will be shown for the entire group.
- Budget: **\$13 total**.

Submission Format

Submit your agent's **Vercel URL** and **GitHub repository link** in the following format:

Vercel URL: {your url}

GitHub Repo URL: {your url}

Deliverable & Deadline

Due date: **23/8/2026**

Final Note

These autonomous agents represent state-of-the-art technology with **advanced reasoning** capabilities. As you develop your agent, think about the broader implications and potential applications across various areas of life. Consider what new possibilities can be unlocked by harnessing these cutting-edge tools to solve real-world problems and enhance our daily experiences. Never it was possible to develop a fully autonomous agent so fast and with such quality - It's pretty cool 😊

Good luck!

Idan Hahn